

Informe de Implementación Arquitectónica

Separación de Experiencias por Roles (Paciente vs Administrador), Navegación Modular y Seguridad en Frontend y Backend

Proyecto: Sistema Hospitalario Lázaro	Fecha: Septiembre 2026
Propósito: Implementación según propuesta PDF de Arquitectura	Estado: 100% Implementado y Verificado

1. Objetivo y Visión General

El objetivo central de este desarrollo fue consolidar **Lázaro** como un sistema único manteniendo dos experiencias de usuario completamente diferenciadas: una **aplicación accesible, intuitiva y orientada a la acción para pacientes**, y un **panel de supervisión y gestión operativo para administradores y personal hospitalario**. Todo ello conservando un único proyecto frontend en React Native, optimizando costos de desarrollo y mantenimiento.

2. Lógica de Navegación Condicional por Roles

Se implementó un flujo de autenticación dinámico mediante AuthContext y AppNavigator.tsx. Tras autenticar las credenciales del usuario, el sistema evalúa el atributo rol para montar exclusivamente el árbol de navegación que le corresponde:

```
// Enrutamiento condicional en AppNavigator.tsx:
if (!usuario) return <AuthNavigator />;
if (usuario.rol === 'ADMINISTRADOR') return <AdminNavigator />;
if (usuario.rol === 'PACIENTE') return <PatientNavigator />;
return <AdminNavigator />; // Médicos / Personal autorizado
```

3. Detalle de Pantallas y Módulos Desarrollados

Módulo / Pantalla	Ubicación de Archivo	Funcionalidad y Rol
Inicio Paciente	screens/patient/PatientHome.tsx	Resumen personal, próxima cita, botón SOS y accesos rápidos.
Mis Citas	screens/patient/PatientAppointments.tsx	Consulta de citas médicas, estado de turnos y solicitud de nueva cita.
Historial Médico	screens/patient/PatientHistory.tsx	Ficha clínica (sangre, alergias) y registro de diagnósticos y observaciones.
Emergencia SOS	screens/patient/Emergency.tsx	Botón de pánico 24/7, transmisión de síntomas críticos y contactos de urgencia.
Directorio Médico	screens/patient/PatientDoctors.tsx	Buscador de especialistas, licencias y enlace para agendar.
Información Hospital	screens/patient/PatientHospitalInfo.tsx	Guía del paciente, horarios de visita, servicios y normativas.
Perfil Paciente	screens/patient/PatientProfile.tsx	Datos personales, logout y conmutador rápido de rol para demo.
Admin Dashboard	screens/admin/AdminDashboard.tsx	KPIs hospitalarios (pacientes, médicos, citas, emergencias, salas).
Especialidades	screens/admin/SpecialtyList.tsx	Módulo CRUD para gestión de especialidades médicas (nueva pantalla).
Gestión Hospitalaria	screens/admin/ (Patient, Doctor, etc.)	CRUDs de Pacientes, Médicos, Citas, Emergencias, Salas, Historiales y Usuarios.
Login & Registro	screens/auth/ (LoginScreen, Register)	Autenticación con backend, creación de cuenta y accesos demo de un clic.

4. Backend: Endpoint de Autenticación y Seguridad

Para respaldar la seguridad en el servidor y no depender únicamente del frontend (Sección 7 del documento base), se actualizó el controlador UsuariosController.cs en **LazaroBackend**:

Endpoint añadido: POST /api/usuarios/login
Validación: Comprueba usuario activo (u.Activo == true), valida credenciales y retorna el objeto con su rol (ADMINISTRADOR, PACIENTE, etc.).
Modo Asistido / Contingencia: El frontend cuenta con soporte dual: conecta automáticamente con la API REST y, si el backend o la base de datos se encuentra fuera de línea, permite el acceso con usuarios demo para garantizar presentaciones y defensas fluidas.

5. Verificación y Calidad de Código

Se ejecutó la verificación estática de tipos con el compilador oficial de TypeScript:

```
.\node_modules\.bin\tsc.cmd --noEmit
```

Resultado: Código 0 (Exitoso), garantizando coherencia absoluta en las interfaces de datos (Medico, Paciente, Cita, HistorialClinico, Especialidad y UsuarioAuth).

6. Conclusión

La arquitectura implementada transforma a **Lázaro** en una plataforma hospitalaria moderna, modular y escalable.

Los pacientes gozan de una aplicación amigable centrada en su salud y seguridad inmediata, mientras que la administración mantiene el control operativo y supervisión total en una única base de código sincronizada.